

Generic HashSet<T> Collection Class in C# with Examples

In this article, I am going to discuss the **Generic HashSet<T> Collection Class in C#** with Examples. Please read our previous article where we discussed [For Each Loop in C#](#) with Examples. At the end of this article, you will understand the following pointers with examples.

1. [What is HashSet<T> in C#?](#)
2. [How to Create a Generic HashSet Collection in C#?](#)
3. [How to Add Elements into a HashSet Collection in C#?](#)
4. [How to Access a Generic HashSet<T> Collection in C#?](#)
5. [Example to Understand How to Create a HashSet and Add Elements in C#](#)
6. [How to Remove Elements from a Generic HashSet<T> Collection in C#?](#)
7. [How to Check the Availability of an Element in a HashSet in C#?](#)
8. [Set Operations on Generic HashSet<T> Collection Class in C#](#)
9. [Generic HashSet Collection with Complex Type in C#](#)
10. [Getting an Enumerator that Iterates through HashSet<T> Collection in C#](#)

What is HashSet<T> in C#?

The Generic HashSet<T> Collection Class in C# can be used to store, remove or view elements. It is an unordered collection of unique elements. The HashSet<T> Collection is introduced in .NET Framework 3.5. It does not allow for the addition of duplicate elements. So, it is recommended to use the HashSet collection if you want to store only unique elements. This collection is of the generic type collection and hence it belongs to System.Collections.Generic namespace. The performance of the HashSet is much better in comparison to the list collection in C#.

How to create a Generic HashSet Collection in C#?

The Generic HashSet Collection class in C# provided seven constructors that we can use to create an instance of HashSet. They are as follows:

1. **public HashSet():** It initializes a new instance of the System.Collections.Generic.HashSet class that is empty and uses the default equality comparer for the set type.
2. **public HashSet(IEnumerable<T> collection):** It initializes a new instance of the System.Collections.Generic.HashSet class that uses the default equality comparer for the set type, contains elements copied from the specified collection and has sufficient capacity to accommodate the number of elements copied.
3. **public HashSet(IEqualityComparer<T> comparer):** It initializes a new instance of the System.Collections.Generic.HashSet class that is empty and uses the specified equality comparer for the set type.
4. **public HashSet(int capacity):** It initializes a new instance of the System.Collections.Generic.HashSet class that is empty, but has reserved space for capacity items and uses the default equality comparer for the set type.
5. **public HashSet(IEnumerable<T> collection, IEqualityComparer<T> comparer):** It initializes a new instance of the System.Collections.Generic.HashSet class that uses the specified equality comparer for the set type, contains elements copied from the specified collection and has sufficient capacity to accommodate the number of elements copied.
6. **public HashSet(int capacity, IEqualityComparer<T> comparer):** It initializes a new instance of the System.Collections.Generic.HashSet class that uses the specified equality comparer for the set type, and has sufficient capacity to accommodate capacity elements.

7. **protected HashSet(SerializationInfo info, StreamingContext context)**: It initializes a new instance of the System.Collections.Generic.HashSet class with serialized data.

Let's see how to create an instance of the HashSet using the HashSet() constructor in C#. The HashSet() is used to create an instance of the HashSet class that is empty and uses the default equality comparer for the set type.

Step1:

As the HashSet<T> class belongs to System.Collections.Generic namespace, so first, we need to import the System.Collections.Generic namespace into our program as follows:

```
using System.Collections.Generic;
```

Step2:

Next, we need to create an instance of the HashSet class using the HashSet() constructor as follows:

```
HashSet<Type_of_hashset> hashSet = new HashSet<Type_of_hashset>();
```

How to Add Elements into a HashSet Collection in C#?

If you want to add elements to your HashSet Collection, then you need to use the following Add() method of the HashSet class.

- **Add(T item)**: This method is used to add the specified element to a set. The parameter item specifies the element to add to the set. It returns true if the element is added to the System.Collections.Generic.HashSet object; false if the element is already present.

The following shows how to add elements using Add method of the HashSet class. Here, you can only store string values and if you try to store other types of values, you will get compile time error.

```
HashSet<string> hashSetCountries = new HashSet<string>();  
hashSetCountries.Add("INDIA");  
hashSetCountries.Add("USA");  
hashSetCountries.Add("UK");
```

Adding Duplicate Elements: Now, if you add duplicate elements into the collection then you will get any compile time error, but as these elements are already added into the collection, hence, it will add them again and, in this way, it makes sure that the collection does not have any duplicate elements.

```
hashSetCountries.Add("USA");  
hashSetCountries.Add("UK");
```

As the HashSet class provides the Add method which accepts an element of type T, so you can also store elements in the Generic HashSet Collection using Collection Initializer as follows.

```
HashSet<string> hashSetCountries = new HashSet<string>  
{  
    "INDIA",  
    "USA",  
    "UK"  
};
```

How to Access a Generic HashSet<T> Collection in C#?

We can access the elements of the HashSet<T> collection in C# using the ForEach loop as follows:

```
foreach (var item in hashSetCountries)  
{
```

```
    Console.WriteLine(item);  
}
```

Note: You cannot use a for loop to access the elements of a generic HashSet collection and the reason for this is the Generic HashSet collection class does not have any integer indexer.

Example to Understand How to Create a HashSet and Add Elements in C#:

For a better understanding of how to create a Generic HashSet<T> collection and how to add elements to a HashSet, and how to access the elements of a HashSet in C# using For Each loop, please have a look at the below example where we created a HashSet of string type. Here, when we are trying to Add duplicate elements then we will not get any compile time error but duplicate elements are not added to the collection. Duplicate elements are simply ignored and, in this way, the Generic HashSet<T> collection maintains the uniqueness of the values.

```
using System;  
using System.Collections.Generic;  
namespace GenericHashSetDemo  
{  
    class Program  
    {  
        static void Main()  
        {  
            //Creating HashSet  
            HashSet<string> hashSetCountries = new HashSet<string>();  
            //Adding Elements to HashSet using Add Method  
            hashSetCountries.Add("INDIA");  
            hashSetCountries.Add("USA");  
            hashSetCountries.Add("UK");  
            //Adding Duplicate Elements will not give compile time error  
            //But duplicate elements are simply ignored and will not be added  
collection  
            hashSetCountries.Add("UK");  
            hashSetCountries.Add("INDIA");  
            Console.WriteLine($"Elements Count: {hashSetCountries.Count}");  
            //Accessing HashSet collection using For Each Loop  
            //Here, you can observe UK and INDIA are printed once  
            Console.WriteLine($"Accessing HashSet Elements:");  
            foreach (var item in hashSetCountries)  
            {  
                Console.WriteLine(item);  
            }  
            Console.ReadKey();  
        }  
    }  
}
```

Output:

```
Elements Count: 3
Accessing HashSet Elements:
INDIA
USA
UK
```

As you can see in the above output, UK and INDIA have printed only once and the count property returns 3 which proves those duplicate elements are not accepted by the Generic HashSet collection class in C#.

Adding Elements to HashSet Collection using Collection Initializer in C#:

In the below example, we are using Collection Initializer syntax instead of the Add method to add elements into the Generic HashSet Collection in C#. The following example will give you the same output as the previous example. Here also the duplicate elements are not accepted by the generic HashSet collection.

```
using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            //Creating HashSet
            HashSet<string> hashSetCountries = new HashSet<string>
            {
                //Adding Elements to HashSet using Add Method
                "INDIA",
                "USA",
                "UK",
                //Adding Duplicate Elements will not give compile time error
                //But duplicate elements are simply ignored and will not be added
                "UK",
                "INDIA"
            };
            Console.WriteLine($"Elements Count: {hashSetCountries.Count}");
            //Accessing HashSet collection using For Each Loop
            //Here, you can observe UK and INDIA are printed once
            Console.WriteLine($"Accessing HashSet Elements:");
            foreach (var item in hashSetCountries)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Elements Count: 3
Accessing HashSet Elements:
INDIA
USA
UK
```

How to Remove Elements from a Generic HashSet<T> Collection in C#?

The Generic HashSet<T> Collection Class in C# provides the following three methods to remove elements from the HashSet.

1. **Remove(T item):** This method is used to remove the specified element from a HashSet object. Here, the parameter item specifies the element to remove. It returns true if the element is successfully found and removed; otherwise, false. This method returns false if the item is not found in the System.Collections.Generic.HashSet object.
2. **RemoveWhere(Predicate<T> match):** This method is used to remove all elements that match the conditions defined by the specified predicate from a HashSet collection. It returns the number of elements that were removed from the HashSet collection. Here, the parameter match specifies the Predicate delegate that defines the conditions of the elements to remove.
3. **Clear():** This method is used to remove all elements from a HashSet object.

Let us see an example to understand the above methods of Generic HashSet Collection Class in C#. Please have a look at the below example where we created a Generic HashSet Collection of string types to store country names.

```
using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            //Creating HashSet and Adding Elements to HashSet using Collection
Initializer
            HashSet<string> hashSetCountries = new HashSet<string>()
            {
                "Bangladesh",
                "Nepal"
            };
            //Adding Elements to HashSet using Add Method
            hashSetCountries.Add("INDIA");
            hashSetCountries.Add("USA");
            hashSetCountries.Add("UK");
            //Adding Duplicate Elements will not give compile time error
            //But duplicate elements are simply ignored and will not be added
collection
            hashSetCountries.Add("UK");
```

```

        hashSetCountries.Add("INDIA");

        Console.WriteLine($"HashSet Elements Count Before Removing:
{hashSetCountries.Count}");
        foreach (var item in hashSetCountries)
        {
            Console.WriteLine(item);
        }
        //Removing element Bangladesh from HashSet Using Remove() method
        hashSetCountries.Remove("Bangladesh");
        Console.WriteLine($"\\nHashSet Elements Count After Removing Bangla
{hashSetCountries.Count}");
        foreach (var item in hashSetCountries)
        {
            Console.WriteLine(item);
        }
        //Remove Element from HashSet Using RemoveWhere() method where ele
is > 3
        int NumberOfElementRemoved = hashSetCountries.RemoveWhere(x => x.Length > 3);
        Console.WriteLine($"\\nHashSet Elements Count After Removeing
{NumberOfElementRemoved} Elements : {hashSetCountries.Count}");
        foreach (var item in hashSetCountries)
        {
            Console.WriteLine(item);
        }
        //Remove All Elements Using Clear method
        hashSetCountries.Clear();
        Console.WriteLine($"\\nHashSet Elements Count After Clear:
{hashSetCountries.Count}");
        Console.ReadKey();
    }
}

```

Output:

```

HashSet Elements Count Before Removing: 5
Bangladesh
Nepal
INDIA
USA
UK

HashSet Elements Count After Removing Bangladesh: 4
Nepal
INDIA
USA
UK

HashSet Elements Count After Removeing 2 Elements : 2
USA
UK

HashSet Elements Count After Clear: 0

```

How to Check the Availability of an Element in a HashSet in C#?

If you want to check whether an element exists or not in the HashSet, then you can use the following Contains() method of the HashSet class.

1. **public bool Contains(T item):** This method is used to determine whether a HashSet object contains the specified element. The parameter item specifies the element to locate in the HashSet object. It returns true if the HashSet object contains the specified element; otherwise, false.

Let us understand this with an example. The following example shows how to use the Contains() method of the Generic HashSet Collection class in C#.

```

using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            //Creating HashSet
            HashSet<string> hashSetCountries = new HashSet<string>();

            //Adding Elements to HashSet using Add Method
            hashSetCountries.Add("INDIA");
            hashSetCountries.Add("USA");
            hashSetCountries.Add("UK");
            //Checking the key using the Contains method
            Console.WriteLine("Is INDIA Key Exists : " +
            hashSetCountries.Contains("INDIA"));
        }
    }
}

```

```

        Console.WriteLine("Is NZ Key Exists : " + hashSetCountries.Contains("NZ"));
        Console.ReadKey();
    }
}

```

Output:

```

Is INDIA Key Exists : True
Is NZ Key Exists : False

```

Set Operations on Generic HashSet<T> Collection Class in C#

The Generic HashSet Collection Class in C# also provides some methods that we can use to perform different set operations. The methods are as follows.

1. **UnionWith(IEnumerable<T> other):** This method is used to modify the current HashSet object to contain all elements that are present in itself, the specified collection, or both. Here, the parameter other specifies the collection to compare to the current HashSet object. If the parameter other is null, then we will get ArgumentNullException.
2. **IntersectWith(IEnumerable<T> other):** This method is used to modify the current HashSet object to contain only elements that are present in that object and in the specified collection. Here, the parameter other specifies the collection to compare to the current HashSet object. If the parameter other is null, then we will get ArgumentNullException.
3. **ExceptWith(IEnumerable<T> other):** This method is used to remove all elements in the specified collection from the current HashSet object. Here, the parameter other specifies the collection of items to remove from the HashSet object. If the parameter other is null, then we will get ArgumentNullException.
4. **SymmetricExceptWith(IEnumerable<T> other):** This method is used to modify the current HashSet object to contain only elements that are present either in that object or in the specified collection, but not both. Here, the parameter other specifies the collection to compare to the current HashSet object. If the parameter other is null, then it will throw ArgumentNullException.

HashSet UnionWith(IEnumerable<T> other) Example in C#:

The UnionWith method contains all the elements which are present in both the collection while removing the duplicate elements. For a better understanding please have a look at the below example where we created two HashSet collections of string type. And then we are calling the UnionWith method on the first HashSet instance and to this method, we are passing the second HashSet collection. Now, the first HashSet collection going to store the elements which are present in both the hash sets by removing the duplicate elements. Here, you can observe that IND and USA are available in both hash sets.

```

using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {

```

```

//Creating HashSet Collection1
HashSet<string> hashSetCountries1 = new HashSet<string>();
//Adding Elements to HashSet using Add Method
hashSetCountries1.Add("IND");
hashSetCountries1.Add("USA");
hashSetCountries1.Add("UK");
hashSetCountries1.Add("NZ");
hashSetCountries1.Add("BAN");
Console.WriteLine("HashSet 1 Elements");
foreach (var item in hashSetCountries1)
{
    Console.WriteLine(item);
}
//Creating HashSet Collection2
HashSet<string> hashSetCountries2 = new HashSet<string>();
//Adding Elements to HashSet using Add Method
hashSetCountries2.Add("IND");
hashSetCountries2.Add("SA");
hashSetCountries2.Add("SL");
hashSetCountries2.Add("USA");
hashSetCountries2.Add("ZIM");
Console.WriteLine("\nHashSet 2 Elements");
foreach (var item in hashSetCountries2)
{
    Console.WriteLine(item);
}
// Using UnionWith method
hashSetCountries1.UnionWith(hashSetCountries2);
Console.WriteLine("\nHashSet 1 Elements After UnionWith");
foreach (var item in hashSetCountries1)
{
    Console.WriteLine(item);
}
Console.ReadKey();
}
}
}

```

Output:

```
HashSet 1 Elements
IND
USA
UK
NZ
BAN

HashSet 2 Elements
IND
SA
SL
USA
ZIM

HashSet 1 Elements After UnionWith
IND
USA
UK
NZ
BAN
SA
SL
ZIM
```

HashSet IntersectWith(IEnumerable<T> other) Example in C#:

The IntersectWith method of the Generic HashSet Collection Class contains the common elements which are present in both the collection. For a better understanding please have a look at the below example where we created two HashSet collections of string type. If you observe both collections, then you will see that IND and USA are the elements that are available in both collections. Then we are calling the IntersectWith method on the first HashSet instance and to this method, we are passing the second HashSet collection. Now, the first HashSet collection going to store the elements which are common in both the hash sets i.e. it is going to contain only the IND and USA elements.

```
using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            //Creating HashSet Collection1
            HashSet<string> hashSetCountries1 = new HashSet<string>();
            //Adding Elements to HashSet using Add Method
            hashSetCountries1.Add("IND");
            hashSetCountries1.Add("USA");
            hashSetCountries1.Add("UK");
            hashSetCountries1.Add("NZ");
        }
    }
}
```

```

hashSetCountries1.Add("BAN");
Console.WriteLine("HashSet 1 Elements");
foreach (var item in hashSetCountries1)
{
    Console.WriteLine(item);
}
//Creating HashSet Collection2
HashSet<string> hashSetCountries2 = new HashSet<string>();
//Adding Elements to HashSet using Add Method
hashSetCountries2.Add("IND");
hashSetCountries2.Add("USA");
hashSetCountries2.Add("SA");
hashSetCountries2.Add("SL");
hashSetCountries2.Add("ZIM");
Console.WriteLine("\nHashSet 2 Elements");
foreach (var item in hashSetCountries2)
{
    Console.WriteLine(item);
}
//Using IntersectWith method
hashSetCountries1.IntersectWith(hashSetCountries2);
Console.WriteLine("\nHashSet 1 Elements After IntersectWith");
foreach (var item in hashSetCountries1)
{
    Console.WriteLine(item);
}
Console.ReadKey();
}
}
}

```

Output:

```
HashSet 1 Elements
IND
USA
UK
NZ
BAN

HashSet 2 Elements
IND
USA
SA
SL
ZIM

HashSet 1 Elements After IntersectWith
IND
USA
```

HashSet ExceptWith(IEnumerable<T> other) Example in C#:

The ExceptWith method of the Generic HashSet Collection Class contains the elements from the first collection which are not present in the second collection. For a better understanding please have a look at the below example where we created two HashSets of string type. If you observe both collections, then you will see that IND and USA are the elements that are available in both collections. Then we are calling the ExceptWith method on the first HashSet instance and to this method, we are passing the second HashSet collection. Now, the first HashSet collection going to store the elements from the first collection except for USA and IND as these two elements are also available in the second collection.

```
using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            //Creating HashSet Collection1
            HashSet<string> hashSetCountries1 = new HashSet<string>();
            //Adding Elements to HashSet using Add Method
            hashSetCountries1.Add("IND");
            hashSetCountries1.Add("USA");
            hashSetCountries1.Add("UK");
            hashSetCountries1.Add("NZ");
            hashSetCountries1.Add("BAN");
            Console.WriteLine("HashSet 1 Elements");
            foreach (var item in hashSetCountries1)
            {
                Console.WriteLine(item);
            }
            //Creating HashSet Collection2
```

```

HashSet<string> hashSetCountries2 = new HashSet<string>();
//Adding Elements to HashSet using Add Method
hashSetCountries2.Add("IND");
hashSetCountries2.Add("USA");
hashSetCountries2.Add("SA");
hashSetCountries2.Add("SL");
hashSetCountries2.Add("ZIM");
Console.WriteLine("\nHashSet 2 Elements");
foreach (var item in hashSetCountries2)
{
    Console.WriteLine(item);
}
//Using ExceptWith method
hashSetCountries1.ExceptWith(hashSetCountries2);
Console.WriteLine("\nHashSet 1 Elements After ExceptWith ");
foreach (var item in hashSetCountries1)
{
    Console.WriteLine(item);
}
Console.ReadKey();
    }
}

```

Output:

```

HashSet 1 Elements
IND
USA
UK
NZ
BAN

HashSet 2 Elements
IND
USA
SA
SL
ZIM

HashSet 1 Elements After ExceptWith
UK
NZ
BAN

```

HashSet SymmetricExceptWith(IEnumerable<T> other) Example in C#:

The SymmetricExceptWith method contains elements that are not common in both collections. For a better understanding please have a look at the below example where we created two HashSet collections of string

type. If you observe both collections, then you will see that IND and USA are the elements that are available in both collections. Here, we are calling the SymmetricExceptWith method on the first HashSet instance and to this method. we are passing the second HashSet collection. Now, the first HashSet collection going to contain the elements which are not common in both HashSets i.e. it is going to contain all elements except the IND and USA elements.

```
using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            //Creating HashSet Collection1
            HashSet<string> hashSetCountries1 = new HashSet<string>();
            //Adding Elements to HashSet using Add Method
            hashSetCountries1.Add("IND");
            hashSetCountries1.Add("USA");
            hashSetCountries1.Add("UK");
            hashSetCountries1.Add("NZ");
            hashSetCountries1.Add("BAN");
            Console.WriteLine("HashSet 1 Elements");
            foreach (var item in hashSetCountries1)
            {
                Console.WriteLine(item);
            }
            //Creating HashSet Collection2
            HashSet<string> hashSetCountries2 = new HashSet<string>();
            //Adding Elements to HashSet using Add Method
            hashSetCountries2.Add("IND");
            hashSetCountries2.Add("USA");
            hashSetCountries2.Add("SA");
            hashSetCountries2.Add("SL");
            hashSetCountries2.Add("ZIM");
            Console.WriteLine("\nHashSet 2 Elements");
            foreach (var item in hashSetCountries2)
            {
                Console.WriteLine(item);
            }
            //Using SymmetricExceptWith method
            hashSetCountries1.SymmetricExceptWith(hashSetCountries2);
            Console.WriteLine("\nHashSet 1 Elements After SymmetricExceptWith");
            foreach (var item in hashSetCountries1)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
HashSet 1 Elements
IND
USA
UK
NZ
BAN

HashSet 2 Elements
IND
USA
SA
SL
ZIM

HashSet 1 Elements After SymmetricExceptWith
SL
SA
UK
NZ
BAN
ZIM
```

Generic HashSet Collection with Complex Type in C#:

As of now, we have used the built-in string type with HashSet. Now, let us proceed further and see how to create a HashSet collection of Complex types. Let us create a class called Student and then let us create a collection of Student types and also add duplicate elements.

```
using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            HashSet<Student> hashSetStudents = new HashSet<Student>()
            {
                new Student(){ ID = 101, Name = "Anurag", Branch="CSE"},
                //Adding Duplicate Element
                new Student(){ ID = 101, Name = "Anurag", Branch="CSE"},
                new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"},
                new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
            };
            Console.WriteLine("HashSet Students List");
            foreach (var item in hashSetStudents)
```

```

        {
            Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch:
{item.Branch}");
        }
        Console.ReadKey();
    }
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

hashSet Students List
ID: 101, Name: Anurag, Branch: CSE
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC

```

We have already discussed that the Generic HashSet<T> Collection Class in C# will not allow duplicates into the collection. But if you observe our output, still we are having duplicate records. To overcome this drawback, we need to implement the IEquatable interface, override Equals, and GetHashCode methods as shown in the below image. Here, we want to check the duplicate elements based on the ID property and hence we are using the ID property inside the Equals and GetHashCode methods.

```

public class Student : IEquatable<Student>
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }

    public bool Equals(Student other)
    {
        return this.ID.Equals(other.ID);
    }
    public override int GetHashCode()
    {
        return this.ID.GetHashCode();
    }
}

```

What is IEquatable<T> Interface?

This Interface provides one method called Equals and this method is used for determining the equality of instances.

So, with the above changes in place, now the HashSet will check the uniqueness of the ID column values and if it found any duplicates then it will remove that record. The complete code is given below.

```

using System;
using System.Collections.Generic;
namespace GenericHashSetDemo
{
    class Program
    {
        static void Main()
        {
            HashSet<Student> hashSetStudents = new HashSet<Student>()
            {
                new Student(){ ID = 101, Name = "Anurag", Branch="CSE"},
                //Adding Dupliocate Element
                new Student(){ ID = 101, Name = "Anurag", Branch="CSE"},
                new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"},
                new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
            };
            Console.WriteLine("HashSet Students List");
            foreach (var item in hashSetStudents)
            {

```

```

        Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch:
{item.Branch}");
    }
    Console.ReadKey();
}
}
public class Student : IEquatable<Student>
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
    public bool Equals(Student other)
    {
        return this.ID.Equals(other.ID);
    }
    public override int GetHashCode()
    {
        return this.ID.GetHashCode();
    }
}
}

```

Output:

```

hashSet Students List
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC

```

Note: While the average time complexity for accessing an element in an array is $O(n)$, where n represents the number of elements in the array, the complexity is just $O(1)$ for accessing a particular element in a HashSet. This makes HashSet a good choice for fast searches and for performing set operations. You can use a list if you would like to store a collection of items in a certain order, and maybe include duplicates as well.

Getting an Enumerator that Iterates through HashSet<T> Collection in C#:

The HashSet<T>.GetEnumerator Method is used to get an enumerator that iterates through a HashSet object. It returns a HashSet<T>.Enumerator object for the HashSet<T> object. For a better understanding, please have a look at the below example.

```

using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {

```

```

//Creating HashSet
HashSet<string> hashSetCountries1 = new HashSet<string>();
//Adding Elements to HashSet using Add Method
hashSetCountries1.Add("IND");
hashSetCountries1.Add("USA");
hashSetCountries1.Add("UK");
hashSetCountries1.Add("NZ");
hashSetCountries1.Add("BAN");
HashSet<string>.Enumerator em = hashSetCountries1.GetEnumerator();
while (em.MoveNext())
{
    string val = em.Current;
    Console.WriteLine(val);
}
Console.ReadKey();
}
}

```

Output:

```

IND
USA
UK
NZ
BAN

```

Points to Remember about Enumerators

1. A For Each statement of the C# language hides the complexity of the enumerators. Therefore, it is recommended to use for each loop instead of directly manipulating the enumerator.
2. The Enumerators in C# can only be used to read the data in the collection, but they cannot be used to modify the underlying collection.
3. Current returns the same object until either MoveNext or Reset is called. MoveNext sets Current to the next element.
4. An enumerator remains valid as long as the collection remains unchanged. If changes are made to the collection, such as adding, modifying, or deleting elements, the enumerator is irrecoverably invalidated and its behavior is undefined.
5. This method is an O(1) operation.

Properties of Generic HashSet Collection Class in C#:

1. **Count**: It returns the number of elements that are contained in the set.
2. **Comparer**: It returns the System.Collections.Generic.IEqualityComparer object that is used to determine equality for the values in the set.

Generic HashSet<T> Collection Class Summary:

1. The Generic HashSet<T> Collection Class implements the ICollection<T>, IEnumerable<T>, IEnumerable, IReadOnlyCollection<T>, ISet<T>, IDeserializationCallback, ISerializable interfaces.
2. It is an unordered collection and hence we cannot sort the elements of HashSet as the order of the element is not defined
3. It does not allow the addition of duplicate elements i.e. the elements must be unique in HashSet.
4. The Generic HashSet<T> Collection provides many mathematical set operations, such as intersection, union, and difference.
5. The capacity of a HashSet collection is the number of elements it can hold.
6. The Generic HashSet <T> in C# is a dynamic collection. That means the size of the HashSet is automatically increased when new elements are added to the collection.
7. As the HashSet <T> is a Generic Collection, so we can only store the same type of elements.

In the next article, I am going to discuss the [Generic SortedList<TKey, TValue> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Generic HashSet<T> Collection Class in C#** with Examples. I hope this Generic HashSet<T> Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.